

utiliser impérativement les feuilles quadrillées fournies pour les schémas simplifiés

Une réponse non justifiée sera considérée comme fausse

On considère une réalisation nommée P9. P9 a le même jeu d'instructions que le Mips-32. C'est une réalisation pipeline à 9 étages : **IFC1**, **IFC2**, **IFC3**, **DEC**, **EXE**, **MEM1**, **MEM2**, **MEM3** et **WBK**.

Dans cette réalisation, les accès à la mémoire s'effectuent en trois cycles. Dans le cycle **DEC** on décode l'instruction et on prépare les opérandes. L'opération et le **calcul de l'adresse de l'instruction** suivante s'effectuent dans le cycle **EXE**. Le cycle **WBK** est similaire au cycle **WBK** du Mips-32.

- 1- Cette organisation du pipeline en 9 étages a-t-elle un impact sur le compilateur ?
- 2- Dans la réalisation Mips-32 étudiée en cours, on observe des dépendances de données d'ordre 1, 2 et 3. Dans la réalisation P9, jusqu'à quel ordre peut-on observer des dépendances de données ?
- 3- Dans Mips-32 il existe 8 *bypass* pour limiter la dégradation des performances due aux dépendances de données. Quels sont les *bypass* dans la réalisation P9 ? Pour chaque *bypass* donner un exemple de suite d'instructions qui illustre la nécessité de ce *bypass*.

On souhaite étudier la performance d'exécution d'une fonction de tri sur cette réalisation. On considèrera deux versions différentes de la même fonction.

Soit la fonction **MaxTab**. Cette fonction est utilisée dans l'algorithme de tri par maximum pour trier un tableau d'entiers. Elle prend en entrée un tableau d'entiers **t**. En parcourant le tableau elle identifie la position de l'élément le plus grand du tableau. Puis, elle place cet élément à la fin du tableau.

<pre>int *MaxTab (int *t, unsigned int size) { unsigned int i = 0; unsigned int max_idx = 0; int max = 0; for (i=0 ; i!=size ; i++) if (t[max_idx] < t[i]) max_idx = i; }</pre>	<pre>max = t [max_idx]; t [max_idx] = t [size-1]; t [size-1] = max; return (t); }</pre>
---	---

Un premier compilateur (non pipeline) a produit une première version de code assembleur pour la boucle principale de cette fonction. On suppose qu'à l'entrée de la boucle le registre R4 contient l'adresse du tableau **t**, R5 l'adresse de fin du tableau **t** (l'adresse qui suit immédiatement celle de la dernière case du tableau) et R6 l'adresse de la case du tableau **t** d'index **max_idx** (la variable **max_idx**). R7 contiendra la valeur de **t[max_idx]**.

Version 1 :

```
_For :  Lw      r7 , 0 (r6 )    ; t[max_idx]
        Lw      r8 , 0 (r4 )    ; t[i]
        Slt     r9 , r7 , r8    ; t[max_idx] < t[i] ?
        Beq     r9 , r0 , _Endif
        Or      r6 , r4 , r0
        _Endif: Addiu  r4 , r4 , 4
        Bne     r4 , r5 , _For
```

- 4- Modifier le code pour qu'il soit exécutable sur P9 et analyser l'exécution de la boucle à l'aide d'un schéma simplifié dans le cas où le branchement échoue.
- 5- Combien de cycles sont-ils nécessaires à l'exécution d'une itération de la boucle dans le cas où le branchement échoue et dans le cas où le branchement réussit ? Calculer le CPI et le CPI-utile en supposant que dans 30% des cas le branchement échoue.
- 6- Optimiser le code de la boucle en modifiant l'ordre des instructions de manière à éviter au maximum les cycles perdus (expliquer votre démarche). Combien de cycles sont-ils nécessaires pour effectuer une itération ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.
- 7- Pour améliorer les performances d'exécution, on décide de dérouler une fois le code de la boucle (traitement de 2 éléments à chaque itération). Proposer un code optimisé après le déroulement. Combien de cycles sont-ils nécessaires pour **traiter un élément** ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.

Les versions plus récentes de l'architecture Mips ont été enrichies avec l'instruction **Movn** (Move if Not Zero). Cette instruction de format R transfère le contenu du registre Rs dans le registre Rd à condition que la valeur du registre Rt ne soit pas égale à zéro. Cette instruction s'exécute dans le pipeline comme une instruction de calcul (la condition est évaluée dans le cycle **EXE**). En utilisant cette instruction on peut écrire une seconde version de la boucle.

Version 2 :

```

_For : Lw      r7 , 0 (r6 )      ; t[max_idx]
       Lw      r8 , 0 (r4 )      ; t[i]
       Slt     r9 , r7 , r8      ; t[max_idx] < t[i] ?
       Movn    r6 , r4 , r9
       Addiu   r4 , r4 , 4
       Bne     r4 , r5 , _For

```

- 8- Après adaptation du code de la version 2 à P9, combien de cycles sont-ils nécessaires à l'exécution d'une itération de la boucle ? Calculer le CPI-utile. Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code.
- 9- Optimiser le code de la boucle en modifiant l'ordre des instructions de manière à éviter au maximum les cycles perdus. Combien de cycles sont-ils nécessaires pour effectuer une itération ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.
- 10- Pour améliorer les performances d'exécution, on décide de dérouler une fois le code de la boucle (traitement de 2 éléments à chaque itération). Proposer un code optimisé après le déroulement. Combien de cycles sont-ils nécessaires pour **traiter un élément** ? Il n'est pas nécessaire de faire un schéma mais d'indiquer simplement les cycles perdus sur le code après optimisation.